

Benchmark: ArrayList vs LinkedList

Análise de Algoritmos e Estruturas de Dados

Universidade Regional de Blumenau – FURB

Prof. Gabriel Castellani de Oliveira

Pull Request: <https://github.com/pedrogodri/trabalho-analise-algoritmo/pull/6>

Branch: <https://github.com/pedrogodri/trabalho-analise-algoritmo/tree/Exercicio-01>

Configuração do experimento:

N = 100000 elementos | Semente aleatória = 42

Questão 1 – Inserção no Final

Comparação entre ArrayList e LinkedList inserindo 100000 elementos ao final. Foram realizados 20 testes; a tabela exibe o tempo médio.

Inserção no final	ArrayList	LinkedList
Tempo médio (ms)	0.617	1.224
Mais rápida	ArrayList (1.98x)	

Impacto da capacidade inicial do ArrayList (10 testes cada):

Capacidade Inicial	Testes	Tempo médio (ms)
10	10	0.580
1.000	10	0.251
100.000	10	0.380

Conclusão: pré-alocar a capacidade exata (100.000) elimina realocações internas, reduzindo cópias do array e, portanto, o tempo total de inserção.

Questão 2 – Inserção em Posição Aleatória

Cada um dos 100000 elementos é inserido em uma posição aleatória válida. 20 testes realizados.

Inserção aleatória	ArrayList	LinkedList
Tempo médio (ms)	159.063	7614.403
Mais rápida	ArrayList (47.87x)	

O ArrayList precisa deslocar os elementos após o ponto de inserção — $O(n)$ —, porém se beneficia de operações de memória em bloco (System.arraycopy). O LinkedList não desloca elementos, mas precisa percorrer a lista para encontrar o índice — também $O(n)$.

Questão 3 – Remoção até Esvaziar

As listas são preenchidas com 100000 elementos e esvaziadas por remoções consecutivas. 20 testes para cada

variante.

Remover primeiro elemento	ArrayList	LinkedList
Tempo médio (ms)	345.105	0.448
Mais rápida	LinkedList (769.85x)	

Remover o primeiro elemento é $O(n)$ para o ArrayList (deslocamento de todos os elementos) e $O(1)$ para o LinkedList (apenas atualização de ponteiro).

Remover último elemento	ArrayList	LinkedList
Tempo médio (ms)	0.287	0.537
Mais rápida	ArrayList (1.87x)	

Remover o último elemento é $O(1)$ para ambas as estruturas, mas o ArrayList se beneficia de melhor localidade de cache.

Questão 4 – Remoção em Índice Aleatório

Listas preenchidas são esvaziadas removendo índices aleatórios. 20 testes.

Remoção aleatória	ArrayList	LinkedList
Tempo médio (ms)	155.289	1987.726
Mais rápida	ArrayList (12.80x)	

O ArrayList sofre deslocamento $O(n)$ por remoção, mas a operação de cópia de memória é altamente otimizada pela JVM. O LinkedList evita deslocamentos, porém paga $O(n)$ para navegar até o índice e tem maior sobrecarga de GC por nó-objeto.

Questão 5 – Acessos Aleatórios (10.000 leituras)

10000 acessos aleatórios a uma lista com 100000 elementos. 20 testes realizados.

Acesso aleatório (10k)	ArrayList	LinkedList
Tempo médio (ms)	0.174	360.627
Mais rápida	ArrayList (2078.07x)	

ArrayList: $\text{get}(i)$ em $O(1)$ – cálculo direto de endereço de memória ($\text{base} + i \times \text{tamanho}$).

LinkedList: $\text{get}(i)$ em $O(n)$ – necessário percorrer i nós a partir da cabeça (ou cauda se $i > n/2$).

Resultado plenamente esperado pela teoria clássica.

Questão 6 – Análise Comparativa Geral

Cenários em que o ArrayList foi mais vantajoso

- Inserção no final (Q1): array contíguo permite alocação amortizada $O(1)$ com boa localidade de cache.
- Remoção do último elemento (Q3): operação $O(1)$ sem deslocamentos.
- Acesso aleatório (Q5): $\text{get}(i)$ em $O(1)$, muito superior ao $O(n)$ do LinkedList.
- Remoção aleatória (Q4): `System.arraycopy` é altamente otimizado pela JVM, superando o LinkedList na prática.

Cenários em que o LinkedList foi mais vantajoso

- Remoção do primeiro elemento (Q3): `removeFirst()` é $O(1)$; o `ArrayList` precisa deslocar todos os $n-1$ elementos restantes.
- Inserção/remoção na cabeça em geral: operação $O(1)$ quando não é necessário percorrer a lista.

Os resultados confirmaram a teoria?

- Q5 – Acesso aleatório: `ArrayList` mais rápida? SIM → teoria $O(1)$ vs $O(n)$ CONFIRMADA.
- Q3 – Remover primeiro: `LinkedList` mais rápida? SIM → teoria $O(1)$ vs $O(n)$ CONFIRMADA.
- De forma geral, os resultados são coerentes com a teoria clássica de complexidade.

Fatores práticos que podem influenciar os resultados

- JIT Compiler: a JVM otimiza código executado frequentemente, podendo mascarar diferenças teóricas.
- Localidade de cache: arrays contíguos exploram melhor os caches L1/L2/L3 do processador.
- Garbage Collector: o `LinkedList` cria um objeto `Node` por elemento; muitos objetos aumentam a pressão no GC e podem causar pausas involuntárias.
- Boxing de primitivos: ambas as listas armazenam `Integer` (objeto encapsulado), não `int` primitivo, adicionando sobrecarga de alocação e desreferenciamento.
- Aquecimento (warm-up) da JVM: as primeiras iterações são mais lentas antes de o JIT compilar o bytecode.
- Carga do sistema operacional: outros processos competem por CPU, memória e barramento durante a execução dos testes.